

File Handling C++

Piyush Pant

MRC



File Stream

- In C++, **file streams** allow programs to perform input/output operations on files.
- A 'file stream' is the medium through which data flows between the program and the file.
- These are similar to console I/O (`cin`, `cout`) but work with external files.

Why we need file handling

- Store data persistently
 - Read/write structured data (e.g., logs, records)
 - Communicate with external systems
-

Types of streams:

Stream Type	Data Flow Direction	Purpose	Class Used
Input Stream	File → Program	Reading from file	<code>ifstream</code>
Output Stream	Program → File	Writing to file	<code>ofstream</code>
Input/Output Stream	File ⇌ Program	Read and write	<code>fstream</code>

Example

```
#include <fstream>
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    ofstream outFile("sample.txt"); // Write (if no file : creates and  
    writes)
```

```
    outFile << "Hello, File!";
```

```
    outFile.close();
```

```
    ifstream inFile("sample.txt"); // Read
```

```
    string word;
```

```
    inFile >> word;
```

```
    cout << word; // Output: Hello,
```

```
    inFile.close();
```

```
    return 0;
```

```
}
```

Using Constructor to Open File

In C++, you can open a file directly when creating a file stream object by passing the **filename** and optionally the **file mode** to the constructor.

```
ifstream input("filename.txt");           // Open for reading
ofstream output("filename.txt");           // Open for writing
fstream file("filename.txt", ios::in | ios::out); // Open for both
```

Example : writing to file

Advantage of opening file with constructor

Concise	No need to call <code>open ()</code> separately
Immediate access	File is ready to use right after object creation
Safer	Encourages initialization at declaration

```
#include <fstream>
#include <iostream>
using namespace std;
```

```
int main() {
    ofstream outFile("output.txt"); // Opens file for writing
    if (!outFile) {
        cout << "Error opening file!";
        return 1;
    }

    outFile << "This is written using constructor.\n";
    outFile.close();
    return 0;
}
```

Examples

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream inFile("output.txt"); // Opens file for reading
    if (!inFile) {
        cout << "File not found!";
        return 1;
    }

    string line;
    while (getline(inFile, line)) {
        cout << line << endl;
    }

    inFile.close();
    return 0;
}
```

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    fstream file("data.txt", ios::in | ios::out | ios::app);
    // Read & append
    if (!file) {
        cout << "Error opening file!";
        return 1;
    }

    file << "Appending using fstream constructor.\n";

    file.seekg(0); // Move read pointer to beginning
    string line;
    while (getline(file, line)) {
        cout << line << endl;
    }

    file.close();
    return 0;
}
```

File Opening Modes in C++

Mode

`ios::in`

`ios::out`

`ios::app`

`ios::ate`

`ios::trunc`

`ios::binary`

Description

Open for reading

Open for writing

Append to end of file, don't allow repositioning

Open and move to end , allows repositioning

Truncate file if it exists

Open in binary mode

Common Combinations

`ios::out | ios::app` → Append mode

`ios::in | ios::binary` → Read binary file

`ios::in | ios::out` → Read and write

Example

```
fstream file("data.txt", ios::out | ios::app);
```

```
file << "Appended line\n";
```

```
file.close();
```

Open Function to Open File



- The `open( )` function is a method provided by `ifstream`, `ofstream`, and `fstream` classes to open files **after** the object is declared.

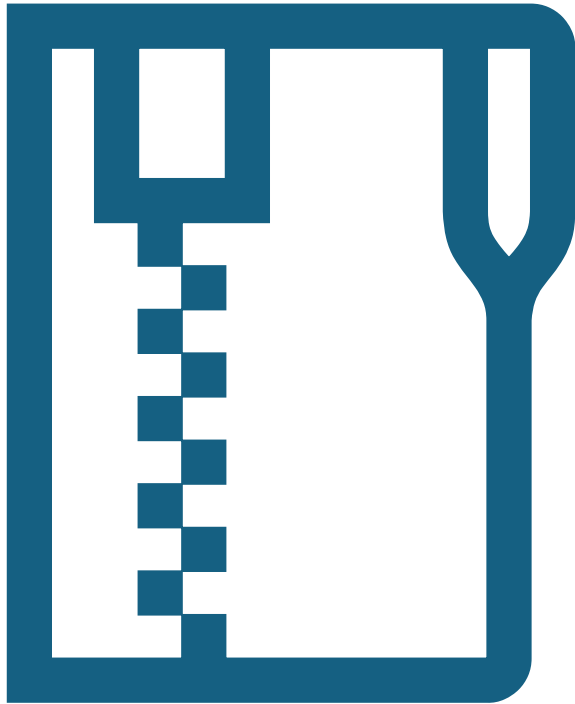
Syntax

```
fstream file;
```

```
file.open("filename.txt", ios::in | ios::out);
```

Note : Practice code from repo .

Sequential and Random Access to File



Sequential access means reading or writing data **in order**, from beginning to end.

Random access allows you to **jump to specific positions** in the file using file pointers.

Refer code from repo